

Bakery Management System

Entity Relationship Diagram Documentation

Version 1.0

NAB Showcase Project

December 2025

Document Information	
Project Name	Bakery Management System
Technology Stack	Spring Boot, Spring Modulith, JPA, MySQL
Total Tables	15
Total Modules	5

Contents

1	System Overview	3
1.1	Business Context	3
1.2	User Roles	3
2	Module Structure	4
2.1	Package Structure	4
3	Module 1: User Management	5
3.1	Table: users	5
3.2	Table: audit_logs	5
4	Module 2: Product Management	7
4.1	Table: products	7
4.2	Table: product_price_history	7
5	Module 3: Storage Management	8
5.1	Table: storage_inventory	8
5.2	Table: storage_transactions	8
6	Module 4: Shift Management	9
6.1	Table: shift_templates	9
6.2	Table: shifts	9
6.3	Table: shift_staff	10
6.4	Table: shift_inventory (CORE AUDIT TABLE)	10
6.5	Table: shift_stock_additions	10
6.6	Table: shift_handovers	11
6.7	Table: shift_handover_details	11
7	Module 5: Order Management (POS)	12
7.1	Table: orders	12
7.2	Table: order_items	12
8	Key Relationships	13
8.1	User Relationships	13
8.2	Product Relationships	13
8.3	Shift Relationships	13
9	Business Flows	14
9.1	Shift Lifecycle	14
9.2	Handover Flow (Double Verification)	14
9.3	Stock Addition Flow	14
9.4	Sales Flow	14
10	Security Considerations (NAB Interview)	15
10.1	Authentication	15
10.2	Audit Trail	15
10.3	Data Integrity	15

1 System Overview

This document describes the database design for a **Bakery Management System** that handles inventory tracking, shift management, sales (POS), and employee management.

1.1 Business Context

- **10 employees** working across 4 shifts daily
- **4 shifts:** Morning (4:00-11:00), Midday (11:00-14:00), Afternoon (14:00-21:00), Night (21:00-4:00)
- **Double verification** on shift handover for accountability
- **Stock flow:** Supplier/Baker → Storage → Shift
- **Payment methods:** Cash and Bank Transfer
- **Full audit trail** for security compliance

1.2 User Roles

Role	Responsibilities
OWNER	Administration only: view reports, manage products, manage staff, manage storage
STAFF	Can be assigned as Shift Manager (rotational), record inventory, handle sales

2 Module Structure

Module	Tables	Purpose
User	users, audit_logs	Authentication, authorization, audit
Product	products, product_price_history	Product catalog, price history
Storage	storage_inventory, storage_transactions	Stock in storage
Shift	shift_templates, shifts, shift_staff, shift_inventory, shift_stock_additions, shift_handovers, shift_handover_details	Shift operations
Order	orders, order_items	POS transactions

2.1 Package Structure

```
com.bakery
|-- user/          -> Module 1
|-- product/       -> Module 2
|-- storage/       -> Module 3
|-- shift/         -> Module 4
|-- order/         -> Module 5
```

3 Module 1: User Management

3.1 Table: users

Purpose: Stores all system users. Used for authentication and authorization.

Business Rules:

- Two roles: OWNER and STAFF
- Username must be unique
- Passwords are BCrypt encrypted
- Soft delete via is_active flag

Column	Type	Null	Description
id	BIGINT	No	Primary key, auto-increment
username	VARCHAR(50)	No	Unique login identifier
password	VARCHAR(255)	No	BCrypt encoded password
full_name	NVARCHAR(100)	No	Display name (Vietnamese supported)
phone	VARCHAR(15)	Yes	Contact phone number
role	VARCHAR(20)	No	OWNER or STAFF
is_active	BOOLEAN	No	Soft delete flag. Default: true
created_at	TIMESTAMP	No	Account creation time
updated_at	TIMESTAMP	Yes	Last modification time
created_by	BIGINT	Yes	FK → users.id

3.2 Table: audit_logs

Purpose: Tracks all important actions for security compliance. Essential for NAB interview.

Business Rules:

- Never deleted, only appended
- Stores JSON snapshots before/after changes
- Records IP address and browser info

Column	Type	Null	Description
id	BIGINT	No	Primary key
user_id	BIGINT	Yes	FK → users.id
action	VARCHAR(50)	No	LOGIN, LOGOUT, CREATE, UPDATE, DELETE
entity_type	VARCHAR(50)	No	SHIFT, ORDER, PRODUCT, etc.
entity_id	BIGINT	Yes	ID of affected record
old_value	TEXT	Yes	JSON before change
new_value	TEXT	Yes	JSON after change
ip_address	VARCHAR(45)	Yes	Client IP (IPv6 supported)

user_agent	VARCHAR(500)	Yes	Browser/device info
created_at	TIMESTAMP	No	When action occurred

4 Module 2: Product Management

4.1 Table: products

Purpose: Master list of all bakery products with current prices.

Business Rules:

- Both selling_price and cost_price required for profit calculation
- Soft delete preserves historical data
- Images stored in AWS S3

Column	Type	Null	Description
id	BIGINT	No	Primary key
name	NVARCHAR(100)	No	Product name (e.g., Banh bao)
selling_price	DECIMAL(10,2)	No	Current selling price
cost_price	DECIMAL(10,2)	No	Cost for profit calculation
image_url	VARCHAR(500)	Yes	AWS S3 URL
is_active	BOOLEAN	No	Soft delete. Default: true
created_at	TIMESTAMP	No	When added
updated_at	TIMESTAMP	Yes	Last modification

4.2 Table: product_price_history

Purpose: Tracks price changes. Old reports show correct historical prices.

Business Rules:

- When price changes: old record gets effective_to = now()
- effective_to = NULL means current price

Column	Type	Null	Description
id	BIGINT	No	Primary key
product_id	BIGINT	No	FK → products.id
selling_price	DECIMAL(10,2)	No	Price during this period
cost_price	DECIMAL(10,2)	No	Cost during this period
effective_from	TIMESTAMP	No	When price became active
effective_to	TIMESTAMP	Yes	When ended. NULL = current
changed_by	BIGINT	No	FK → users.id

5 Module 3: Storage Management

Only Owner has access. Stock flow: Supplier/Baker → Storage → Shift.

5.1 Table: storage_inventory

Purpose: Current stock levels in storage. One row per product.

Column	Type	Null	Description
id	BIGINT	No	Primary key
product_id	BIGINT	No	FK → products.id (UNIQUE)
quantity	INTEGER	No	Current stock. Default: 0
last_updated	TIMESTAMP	No	When last changed

5.2 Table: storage_transactions

Purpose: Audit trail of stock movements in/out of storage.

Business Rules:

- IN = stock added (from baker or supplier)
- OUT = stock sent to shift
- reference_id links to shift_stock_additions when type=OUT

Column	Type	Null	Description
id	BIGINT	No	Primary key
product_id	BIGINT	No	FK → products.id
transaction_type	VARCHAR(20)	No	IN or OUT
quantity	INTEGER	No	Always positive
reference_id	BIGINT	Yes	FK → shift_stock_additions.id
note	NVARCHAR(500)	Yes	Optional explanation
created_by	BIGINT	No	FK → users.id
created_at	TIMESTAMP	No	When occurred

6 Module 4: Shift Management

Core module for shift operations, inventory tracking, and handover verification.

6.1 Table: shift_templates

Purpose: Defines the 4 daily shift time slots.

Column	Type	Null	Description
id	BIGINT	No	Primary key
name	NVARCHAR(50)	No	Morning, Midday, Afternoon, Night
start_time	TIME	No	E.g., 04:00:00
end_time	TIME	No	E.g., 11:00:00
is_active	BOOLEAN	No	Default: true

6.2 Table: shifts

Purpose: Actual shifts worked. Contains cash management and revenue totals.

Business Rules:

- Status: PENDING → OPEN → CLOSED
- One manager per shift (rotational)
- $\text{cash_discrepancy} = \text{actual_cash_handed} - \text{expected_cash}$
- Negative = money missing

Column	Type	Null	Description
id	BIGINT	No	Primary key
shift_template_id	BIGINT	No	FK → shift_templates.id
shift_date	DATE	No	Date of shift
manager_id	BIGINT	No	FK → users.id
status	VARCHAR(20)	No	PENDING, OPEN, CLOSED
opened_at	TIMESTAMP	Yes	When opened
closed_at	TIMESTAMP	Yes	When closed
initial_cash	DECIMAL(12,2)	No	Starting cash. Default: 0
total_cash_sales	DECIMAL(12,2)	No	Cash payments sum
total_transfer_sales	DECIMAL(12,2)	No	Bank transfers sum
total_revenue	DECIMAL(12,2)	No	cash + transfer
total_cost	DECIMAL(12,2)	No	Cost of goods sold
expected_cash	DECIMAL(12,2)	Yes	initial + cash_sales
actual_cash_handed	DECIMAL(12,2)	Yes	Physical count
cash_discrepancy	DECIMAL(12,2)	Yes	actual - expected
cash_note	NVARCHAR(500)	Yes	Explanation
created_at	TIMESTAMP	No	Created time
updated_at	TIMESTAMP	Yes	Modified time

6.3 Table: shift_staff

Purpose: Which staff worked each shift (2-4 per shift typical).

Column	Type	Null	Description
id	BIGINT	No	Primary key
shift_id	BIGINT	No	FK → shifts.id
user_id	BIGINT	No	FK → users.id

6.4 Table: shift_inventory (CORE AUDIT TABLE)

Purpose: Tracks inventory per product per shift.

Key Formulas:

- $\text{system_closing_qty} = \text{opening_qty} + \text{added_qty} - \text{sold_qty}$
- $\text{discrepancy} = \text{actual_closing_qty} - \text{system_closing_qty}$
- Negative discrepancy = LOSS

Column	Type	Null	Description
id	BIGINT	No	Primary key
shift_id	BIGINT	No	FK → shifts.id
product_id	BIGINT	No	FK → products.id
opening_qty	INTEGER	No	From previous shift
opening_confirmed	BOOLEAN	No	Manager confirmed
opening_confirmed_at	TIMESTAMP	Yes	When confirmed
added_qty	INTEGER	No	Mid-shift additions
sold_qty	INTEGER	No	Quantity sold
system_closing_qty	INTEGER	Yes	Calculated closing
actual_closing_qty	INTEGER	Yes	Physical count
discrepancy	INTEGER	Yes	actual - system
discrepancy_note	NVARCHAR(500)	Yes	Explanation
selling_price_snapshot	DECIMAL(10,2)	No	Price at shift time
cost_price_snapshot	DECIMAL(10,2)	No	Cost at shift time

6.5 Table: shift_stock_additions

Purpose: Stock added during shift from storage.

Column	Type	Null	Description
id	BIGINT	No	Primary key
shift_id	BIGINT	No	FK → shifts.id
product_id	BIGINT	No	FK → products.id
quantity	INTEGER	No	Items received
added_by	BIGINT	No	FK → users.id

note	NVARCHAR(500)	Yes	Optional note
created_at	TIMESTAMP	No	When recorded

6.6 Table: shift_handovers

Purpose: Handover between shifts with double verification.

Status Flow:

- PENDING: Shift A closed, waiting for Shift B
- CONFIRMED: Shift B accepted quantities
- DISPUTED: Quantities don't match

Column	Type	Null	Description
id	BIGINT	No	Primary key
from_shift_id	BIGINT	No	FK → shifts.id (closing)
to_shift_id	BIGINT	No	FK → shifts.id (opening)
handed_by	BIGINT	No	FK → users.id
handed_at	TIMESTAMP	No	When initiated
received_by	BIGINT	No	FK → users.id
received_at	TIMESTAMP	Yes	When confirmed
status	VARCHAR(20)	No	PENDING, CONFIRMED, DISPUTED
dispute_note	NVARCHAR(500)	Yes	Explanation if disputed

6.7 Table: shift_handover_details

Purpose: Per-product handover details.

Column	Type	Null	Description
id	BIGINT	No	Primary key
handover_id	BIGINT	No	FK → shift_handovers.id
product_id	BIGINT	No	FK → products.id
handed_qty	INTEGER	No	Qty by closing shift
received_qty	INTEGER	Yes	Qty by opening shift
has_discrepancy	BOOLEAN	No	Default: false
discrepancy_note	NVARCHAR(500)	Yes	Explanation

7 Module 5: Order Management (POS)

7.1 Table: orders

Purpose: Each sale transaction.

Business Rules:

- Order number format: ORD-YYYYMMDD-NNNN
- Payment: CASH or TRANSFER
- Updates shift_inventory.sold_qty automatically

Column	Type	Null	Description
id	BIGINT	No	Primary key
shift_id	BIGINT	No	FK → shifts.id
order_number	VARCHAR(20)	No	Unique identifier
subtotal	DECIMAL(12,2)	No	Before discount
discount	DECIMAL(12,2)	No	Discount. Default: 0
total_amount	DECIMAL(12,2)	No	subtotal - discount
payment_method	VARCHAR(20)	No	CASH or TRANSFER
created_by	BIGINT	No	FK → users.id
created_at	TIMESTAMP	No	When placed

7.2 Table: order_items

Purpose: Line items with price snapshots.

Column	Type	Null	Description
id	BIGINT	No	Primary key
order_id	BIGINT	No	FK → orders.id
product_id	BIGINT	No	FK → products.id
quantity	INTEGER	No	Items sold
unit_price	DECIMAL(10,2)	No	Price at sale time
unit_cost	DECIMAL(10,2)	No	Cost at sale time
line_total	DECIMAL(12,2)	No	quantity x unit_price

8 Key Relationships

8.1 User Relationships

- `users.created_by` → `users.id` (self-reference)
- `users.id` ← `audit_logs.user_id` (1:N)
- `users.id` ← `shifts.manager_id` (1:N)

8.2 Product Relationships

- `products.id` ← `product_price_history.product_id` (1:N)
- `products.id` ↔ `storage_inventory.product_id` (1:1)
- `products.id` ← `order_items.product_id` (1:N)

8.3 Shift Relationships

- `shift_templates.id` ← `shifts.shift_template_id` (1:N)
- `shifts.id` ← `shift_inventory.shift_id` (1:N)
- `shifts.id` ← `orders.shift_id` (1:N)
- `shifts.id` ← `shift_handovers` (links two shifts)

9 Business Flows

9.1 Shift Lifecycle

1. Owner creates shift → PENDING
2. Manager opens → confirms handover → OPEN
3. During shift: sales, stock additions
4. Manager closes → counts inventory & cash → CLOSED
5. System calculates discrepancies

9.2 Handover Flow (Double Verification)

1. Shift A closes → handover PENDING
2. Shift B opens → sees quantities
3. Match → CONFIRMED
4. Mismatch → DISPUTED with note

9.3 Stock Addition Flow

1. Manager needs more stock
2. Records addition → shift_stock_additions
3. System updates shift_inventory.added_qty
4. System creates storage_transactions (OUT)

9.4 Sales Flow

1. Staff creates order
2. System snapshots prices to order_items
3. System updates shift_inventory.sold_qty
4. System updates shifts totals

10 Security Considerations (NAB Interview)

10.1 Authentication

- BCrypt password encryption
- Role-based access (OWNER, STAFF)
- Spring Security integration

10.2 Audit Trail

- All changes logged to audit_logs
- JSON snapshots before/after
- IP and User-Agent tracking
- Append-only design

10.3 Data Integrity

- Price snapshots preserve history
- Double verification on handovers
- Foreign keys maintain consistency
- Soft deletes preserve audit trail

— End of Document —